

Group Project: Database For the Construction Submission Portal

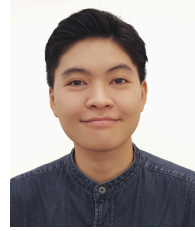
The Team



Alexander Beck
CS BS/MS



Tina Cheng
CS Bachelor



Tran Man Tuyen Dinh
Master of CS



Zongyu Mu
Master of CS

Abstract

This project is the design and implementation of a robust Relational Database Management System (RDBMS) application for a centralized Construction Submission Portal. The system aims to modernize the construction permit process for Commercial Mixed-Use Developments by replacing inefficient paper-based workflows with a digital platform. This centralized hub allows design consultants to securely submit building plans and technical documents, enabling complex, parallel reviews by multiple agency bodies to reduce administrative bottlenecks and version control issues. Our technical approach involves engineering a highly normalized database with 20+ relational tables using PostgreSQL, supported by a FastAPI and SQLAlchemy backend. This stack facilitates seamless database integration, handles file management, and enforces data integrity constraints through FastAPI validation. Major milestones accomplished include the setup of the entire tech stack framework (frontend, backend, database). We have successfully normalized the database schema, implemented the tables in PostgreSQL, established basic authentication, and connected all components. We are roughly 50% complete, having implemented the login/registration logic and the project table view.

Keywords: Relational Database Management System (RDBMS), Construction Submission Portal, Construction Permit Process, Database Normalization, PostgreSQL, FastAPI, SQLAlchemy, Parallel Agency Review, Version Control, Data Integrity Constraints, Submission Packages, Commercial Mixed-Use Developments.

Overview

The nature of this project falls strictly under the Database Application category. We are designing, developing, and implementing a fully functional RDBMS database application to power a real-world web application. Specifically, this project is a centralized platform designed to modernize the construction permit process. The core technical focus is engineering a robust relational database that allows design consultants to securely submit building plans - including 3D models and technical documents - to a central hub, enabling a complex workflow where multiple agency bodies can review, issue feedback, and approve the submissions in parallel.

Background

The construction permitting process in many jurisdictions continues to rely on paper-based submissions, fragmented communication, and sequential agency reviews. This traditional workflow often leads to duplicated documentation, lost records, limited transparency, and prolonged approval timelines. According to the National Association of Home Builders (NAHB, 2021), regulatory costs account for approximately 23.8% of the price of a new home in the United States, highlighting the significant economic impact of regulatory inefficiencies. Furthermore, the construction industry remains one of the least digitized major industries globally (McKinsey Global Institute, 2017), despite evidence that digital transformation initiatives have the potential to improve productivity by 14–15% (McKinsey & Company, 2016).

These challenges underscore the need for a centralized and structured digital permit management system. We are targeting the Commercial Mixed-Use Developments (Condos + Retail) projects. These buildings have the most complex permitting requirements. They involve multiple agencies (fire safety, commercial zoning, residential codes), making a centralized RDBMS incredibly valuable for parallel reviews. For example, in real life there are several problems happened during the building permit process that a RDBMS can help to resolve:

- **Administrative Bottlenecks:** Large residential complexes often require sequential reviews from structural, electrical, and plumbing boards. A digitized, parallel review system prevents the plumbing review from being blocked while waiting for the structural paper file to be physically mailed.
- **Version Control Nightmares:** In mixed-use retail/residential builds, floor plans change frequently. Traditional paper workflows lead to agencies reviewing outdated "Version 1" plans while "Version 3" is already drafted. Our system's versioning ensures all Processing Officers view the single source of truth.
- **The Cost of Delay:** As regulatory costs account for 23.8% of a new home's price, delays in high-density condo approvals cost developers thousands of dollars a day in interest. Enforceable data integrity constraints prevent incomplete submissions, reducing the back-and-forth "Written Directions" and speeding up final approval.

In summary, a database-driven solution can support standardized submissions, parallel agency review, real-time status tracking, and enforceable data integrity constraints. By leveraging relational database management principles, the proposed system aims to streamline coordination among stakeholders, reduce administrative delays, and enhance transparency throughout the permitting lifecycle.

How we address the topic

User Requirements

Projects: The database acts as a parent container for the entire building lifecycle by storing project information. Each project has an auto-generated project reference number, project title, Development Type, Building Works, and an Address. Each project contains multiple project members and multiple submission packages. A project must have exactly one active Project Coordinator (PC) and exactly one Developer.

Project Coordinators (PC): The primary administrator who creates the project and assembles the project team. A single PC can manage multiple projects simultaneously. The system records their Name, Role, Professional Registration number, National ID, and email.

Qualified Persons (QP): Licensed professionals assigned to technical scopes. A QP can be assigned to multiple projects, and within a single project, they can be assigned multiple responsibilities or Sections to declare regulatory compliance. The database stores their Name, Role, Registration number, National ID, and email. Their project responsibility data includes their scope of responsibilities for at least one or many agencies, and whether they prepare plans or supervise work. Additionally, a QP can be assigned to manage a specific submission package, taking on the responsibility of the Submission Coordinator (SC).

Developers: The client entity that legally authorizes the project team, authorizes the submission packages. A Developer can own multiple projects and can be either an Organization or an Individual.

- For an Individual, the database records their Name, National ID, and email.
- For an Organization, the database records the Role, Business Registration Number, and a Main acting personnel containing a Name, National ID, and email. The organization can also have up to 2 secondary developer personnel.

Agency Bodies & Staff: The database stores information for agency bodies (e.g., Building Authority) that review submissions and approve it. Each agency body has a Department Head and multiple Processing Officers (PO). Agency staff details include their name, National ID, phone number, and email address. Each PO is expert on one or multiple project types, and their expertise may overlap with that of other POs. This information about PO must also be recorded. After a submission has been submitted, POs will be assigned to each section.

Submission Packages: A specific application instance sent to the government for review and approval. The PC creates a new submission package and assigns a Submission Coordinator (SC). Each submission package belongs to exactly one Project and contains multiple Sections. The database records the submission ID, submission category, submission type, creation timestamp, version number, and current status (Draft, Pending declaration, Pending Authorization, Pending Payment, Submitted, Processing, WD Issued, Approved, Rejected). When a Written Direction (WD) is replied to, the system generates a new version (e.g., Version 2) of the submission to track revisions. Each Submission Package must be managed by exactly one SC. While a package is managed by one SC, an SC can manage multiple submissions.

Sections: Each submission has one or multiple sections. Each section belongs to exactly one Submission Package, targets one Agency, and is assigned to one QP. A section contains multiple documents. The system records the section ID, agency target, completion status(Incomplete, Ready for Review, Declared, WD Unlocked), and the version.

Documents: Technical files (like PDF plans, report or 3D models) and other legal documents uploaded for review. Each document belongs to exactly one Section. The database records the document ID, document version, file name, file type, upload timestamp, the User ID of who uploaded it, and a read-only lock flag.

Written Directions (WD): Feedback records created when submissions require corrections. Each WD is issued by one Processing Officer (PO) and is linked to one or multiple specific Sections. The database stores the WD ID, issue timestamp, comment text, and severity/type.

Users' queries/questions

By developing our application, we aim to help users to answer these questions:

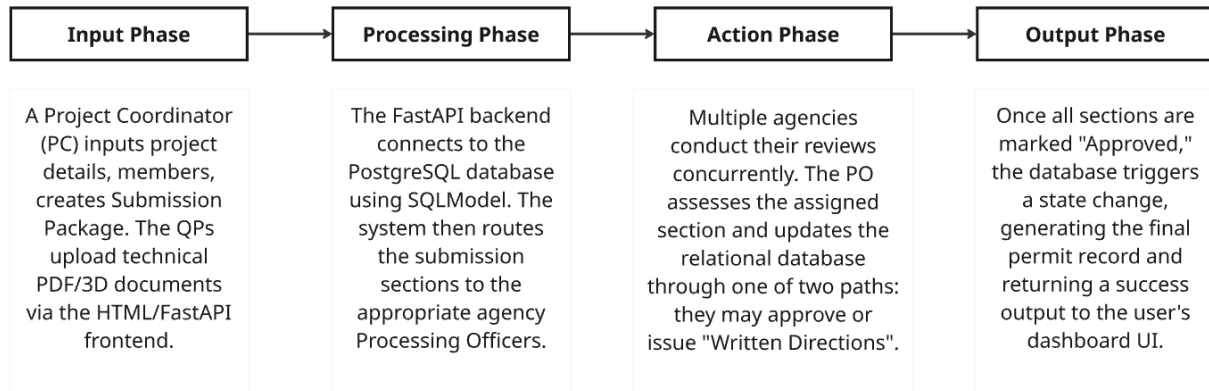
- List all the projects and their current status managed by a specific Project Coordinator.
- List all the submissions and current status of submissions that each Submission Coordinator manages.
- Retrieve all pending actionable tasks (e.g., "Pending Declaration", "Pending Payment", "Pending WD Reply") assigned to a specific user (QP or SC) across all active projects.
- List out all the project members (QPs, Developers) associated with a specific Project ID, including their assigned roles and their current appointment status.
- Retrieve all Submission Packages associated with a specific Project ID.
- List all "Draft" submission packages that were created more than 30 days ago.
- Retrieve the latest version number of a specific Submission Package alongside the current completion status (e.g., Incomplete, Ready for Review, Declared) of every Section it contains.
- List out all the submission packages that have received a final "Approved" or "Rejected" status within a specified date range.
- List out all the "Processing" submissions currently assigned to a specific agency Processing Officer.
- List all distinct Agency Bodies involved in the current active Submission Package for a specific Project ID, along with their individual section approval statuses.
- Retrieve all Written Directions issued for a specific Submission Package, ordered by severity and issue timestamp.
- Count the total number of Written Directions issued by each Processing Officer for a specific Project Type within the last quarter.

Inputs and outputs to and from our product

Inputs: User profile data, role attributes (Qualified Person/Project Coordinator/Developer), registration numbers, digital construction plans, agency assignment data, reviewer decisions and comments, and review statuses.

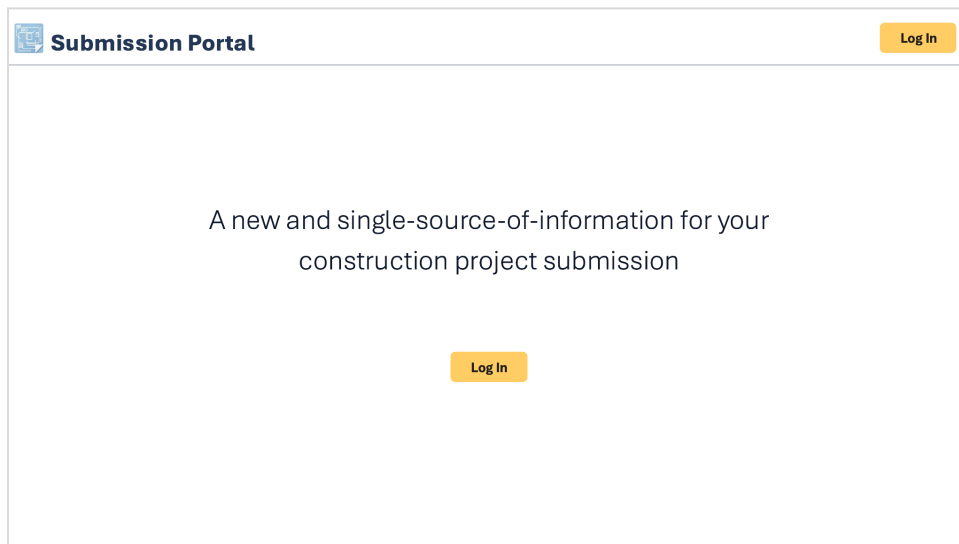
Outputs: Database transaction logs, real-time submission status, reviewer comments, automatically generated permit records, and agency notification records.

High-Level Solution Processes



Users' interface design

Below is the example of the UI design would be implemented (Please refer to the file we attach for more details):




Submission Portal

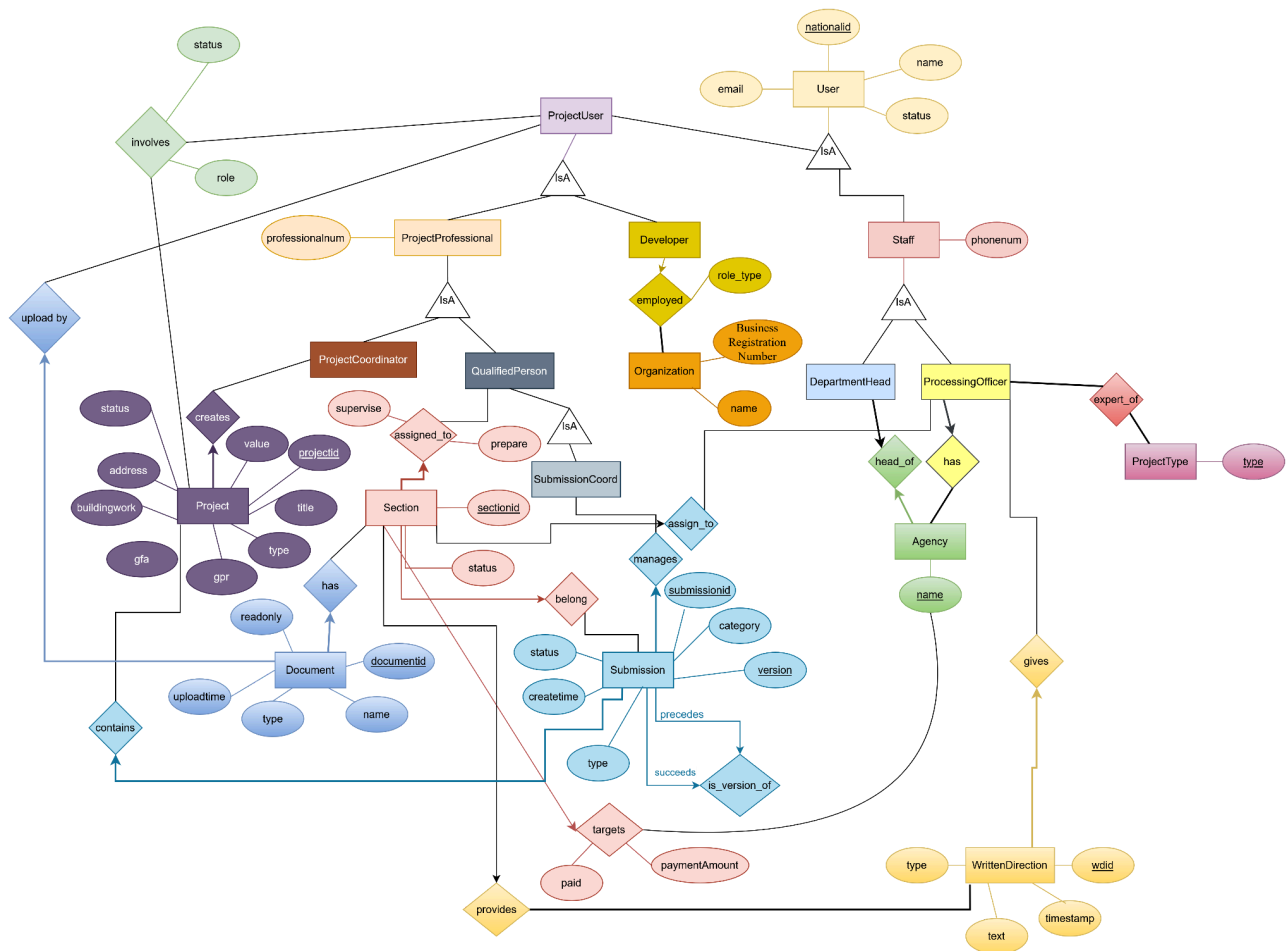
Log in to Submission Portal

Email

Password

ERD database design

Please refer to the file we attach for better quality.



Please refer to the file we attach for better quality.

Page 6

5	Get all users (GET /users/)	HTTP GET to /users/	200 OK	Pass	Mar 29, 2026
6	Database connection	engine.connect()	Connection object not None	TBD	TBD
7	Insert a valid user record	email = "user1@gmail.com", role = "qp"	Record inserted successfully	TBD	TBD
8	Insert duplicate email	email = "user1@gmail.com" (already exists)	UNIQUE constraint violation	TBD	TBD
9	Insert user with NULL email	email = NULL	NOT NULL constraint violation	TBD	TBD
10	Insert valid submission package	projectid = 101, PC nationalid = 1, SC nationalid = 5	Submission created, status = "Draft"	TBD	TBD
11	Insert submission with invalid nationalid	PC nationalid not exist	FOREIGN KEY violation	TBD	TBD
12	Assign ProcessingOfficer to Section	submissionid = 55, sectionid = 3, PO nationalid = 7	Assignment created successfully	TBD	TBD
13	Assign PO with invalid submissionid	submissionid = 8888 (not exist)	FOREIGN KEY violation	TBD	TBD
14	Update Section status to "Approved"	section_status = "Approved"	Status updated successfully	TBD	TBD
15	Insert invalid submission_status	"InvalidStatus"	CHECK constraint violation	TBD	TBD
16	Generate permit after full approval	All Sections = "Approved"	Permit created & linked	TBD	TBD
17	Generate permit before approval	Some Sections = "Pending"	Permit NOT generated	TBD	TBD

18	Delete submission with existing Sections	submission referenced	Deletion restricted	TBD	TBD
19	Insert user with invalid role	role_type = "admin"	CHECK constraint violation	TBD	TBD
20	Insert QualifiedPerson without professionalNum	professionalNum = NULL	NOT NULL constraint violation	TBD	TBD
21	Insert Organization without BRN	business_registration_number = NULL	NOT NULL constraint violation	TBD	TBD
22	QP submits without SC role	nationalid = 10 (not SC)	Permission error	TBD	TBD

Tech stack - Programming/database libraries to be used

Frontend

Due to the nature of the course, the majority of the efforts for this project will be put into the database itself. However, other aspects need to be worked on as well, such as the frontend and file management system. The frontend will be written using basic HTML and CSS, and little time will be allocated to it. It will only be worked on if there is an abundance of time at the end of the project. Tailwind will also be added to make the implementation of CSS easier (Tailwind Labs, N.D.).

The main component that need to be included in the frontend are as follows:

1. A table for viewing the output
2. A button to upload files
3. Login page

Backend

The backend will be more sophisticated than the frontend. It will be responsible for allowing the user to upload and download files, have login capabilities, and connect to the database.

To handle these features, we will use FastAPI (Ramírez, S, *FastAPI*, N.D.). FastAPI is a Python library that allows for easy file sharing, API endpoints, and integration with a SQL database (with SQLAlchemy). Best of all, it is built with the mindset of type hints in mind, which makes it easy to debug. Login management will also be handled by FastAPI, as the documentation includes multiple examples on authentication (Ramírez, S, *FastAPI*, N.D.). Furthermore, another tool by the creators of FastAPI, SQLAlchemy, works seamlessly with FastAPI to create database connections (Ramírez, S, *SQLAlchemy*, N.D.).

Database

For our database, we decided to use PostgreSQL, as it is a common choice in the industry (PostgreSQL Global Development Group, N.D.).

Supplementary Materials

In addition to the above listed sources for the project, we will be following some tutorials and guides to allow us to focus our attention on the DBMS portion of the project. One such tutorial was found on Medium, and it describes how to connect Python to Postgres using FastAPI (Kim, K, 2023). Another document shows how to handle uploading files through FastAPI (Ulili, S, 2025). If there is time, one feature that we would like to implement is allowing users to view the files they upload directly in the browser. It would be restricted to PDFs and 3D files, to keep things simple. To handle the 3D files, we found a guide that makes viewing 3D files easy (Pablo, 2025).

How we document our program codes

We document our code by focusing on readability first, using clear and descriptive names for all tables, columns, and variables so the code naturally explains what it is doing. We rely on high-level external documentation, such as our ERD and a project README, to provide a clear overview of the database structure and system architecture before anyone even looks at the raw code.

List of Problem

1. Because we use sqlalchemy to make the table for our database, it was difficult to integrate triggers and constraint in pure PostgreSQL
2. Setting up async PostgreSQL sessions with SQLAlchemy was challenging because the sync engine caused conflicts with FastAPI's async request lifecycle, requiring a full refactor of the session dependency injection pattern.
3. Normalizing a database was much more difficult than we expected for a database with 20 relational tables.
4. Implementing JWT token expiry and refresh logic was complex — we needed to handle token expiration checks server-side, return proper HTTP 401 responses, and ensure the frontend was redirected correctly on expiry.
5. File upload validation was problematic without proper checks. Without enforcing MIME type and file size limits early in the request pipeline, invalid files could reach storage, causing downstream errors.

Lessons Learned

1. We can do triggers and constraint by writing validation through fastapi
2. SQLAlchemy's inheritance system requires explicit table=True flags on each subtype model; relying on inheritance alone causes ORM mapping conflicts that are difficult to debug without understanding how SQLAlchemy maps Python class hierarchies to relational tables.
3. Split up the normalization process instead of doing it all at once.
4. Designing 20+ normalized tables from the start prevented data redundancy but required careful planning of FK relationships and inheritance hierarchies.
5. Learned to leverage Pydantic type hints through SQLAlchemy for both DB models and API schemas simultaneously, reducing boilerplate significantly.
6. Implementing stateless JWT auth taught the team about token lifecycle, refresh strategies, and secure password hashing with bcrypt.
7. Transitioning to async database sessions required refactoring dependency injection patterns and understanding FastAPI lifespan context managers.
8. Established feature-branch workflows with pull request reviews to coordinate across 4 team members without merge conflicts.

Conclusion

We have finished the setup for the backend, frontend, and database framework, normalized the database, and implemented the tables in postgresQL schema. We have also defined the constraints/triggers through fastapi. Connect the backend to the database, and the backend to the frontend. We also implement basic authentication and the frontend and backend logic for login and registering accounts. We have also made the project table view and the landing page.

We are a bit behind our original timeline as we were not able to implement project team features and create project button form because of the complexity of the setup, it pushed back everything on the timeline but we still accomplished roughly 50% of the task and are still on track to complete this project on time.

The following is the list of tasks that still needs to be done:

Dashboard
Create Project Button
Implement Project Team features
Submission Form
Validation Logic for the Submission Form
File Upload Interface
Implement File Upload API
Implement File Download API
WD comment/display
Review Page Layout
Status Update & Validation
Role-Based UI Restrictions
Cancel & Back Navigation
Unit Testing
Documentation

Timeline

Task	Wk 1 (Mar 30)	Wk 2 (Apr 6)	Wk 3 (Apr 13)	Wk 4 (Apr 20)	Wk 5 (Apr 27)
Finalize ERD & Schema					
Complete Backend CRUD					
Frontend Pages (Login/Dashboard)					
File Upload Integration					
Agency Review Workflow					
API Integration Tests					
End-to-End Testing					
Final Demo & Submission					

Deliverable

The final deliverable will be a demo created based on ERD and database solutions. We will evaluate our final deliverable by ensuring we meet all the user requirements, pass all the test cases, and ensure the usability of the application by testing all features of the application. In addition to our backend SQL test cases, we will conduct User Acceptance Testing (UAT). We will ask 2-3 classmates or friends to roleplay as Project Members and Processing Officers to click through our frontend UI. We will measure success by seeing if they can successfully create a project, upload a document, and approve a submission without encountering unhandled backend API errors.

References

Ataide, M., Braholli, O., & Siegele, D. (2023). Digital transformation of building permits: Current status, maturity, and future prospects. *Buildings*, 13(10), 2554. <https://doi.org/10.3390/buildings13102554>

Kim, K. (2023). *A Guide to Connecting PostgreSQL and Pythons' Fast API: From Installation to Integration*. <https://medium.com/@kevinkoech265/a-guide-to-connecting-postgresql-and-pythons-fast-api-from-installation-to-integration-825f875f9f7d>

McKinsey Global Institute. (2017). *Digital America: A Tale of the Haves and Have-Mores*. <https://www.mckinsey.com/mgi/overview/in-the-news/digital-america-a-tale-of-the-haves-and-have-mores>

McKinsey & Company. (2016). *Imagining construction's digital future*. <https://www.mckinsey.com/capabilities/operations/our-insights/imagining-constructions-digital-future>

National Association of Home Builders. (2021). *Government regulation in the price of a new home*. <https://www.nahb.org/news-and-economics/housing-economics-plus/special-studies/special-studies-pages/government-regulation-in-price-of-new-home-in-2021>

Pablo. (2025). *How to Integrate 3D Models into Your Website Using Only HTML and CSS*. <https://dev.to/checode/how-to-integrate-3d-models-into-your-website-using-only-html-and-css-lok>

PostgreSQL Global Development Group. PostgreSQL [Computer software]. <https://www.postgresql.org/>

Ramírez, S. FastAPI [Computer software]. <https://github.com/fastapi/fastapi>

Ramírez, S. SQLAlchemy [Computer software]. <https://github.com/fastapi/sqlmodel>

Tailwind Labs. Tailwind CSS [Computer software]. <https://github.com/tailwindlabs/tailwindcss>

Ullrich, S. (2025). *Uploading Files Using FastAPI: A Complete Guide to Secure File Handling*. <https://betterstack.com/community/guides/scaling-python/uploading-files-using-fastapi/>